Memory Management 2

Memory Abstractions and Ownership

A game!

Motivation for the Rule Of One

- Mutability is dangerous
- Change and observation don’t mix
- Also explains why everything is immutable by default in Rust!
- How could we safely solve our game using Rust code?

Modeling our game

```rust
fn count_whites(board: &Vec<bool>) -> usize {
    let mut count = 0;
    for piece in board {
        if *piece {
            count += 1;
        }
    }
    count
}

fn scramble(board: &mut Vec<bool>) {
    const ITERATIONS: usize = 256;
    for _ in 0..ITERATIONS {
        let (first, second) = choose_swap_indices();
        board.swap(first, second);
    }
}
```

Modeling our game

```rust
fn count_whites(board: &Vec<bool>) -> usize {
    let mut count = 0;
    for piece in board {
        if *piece {
            count += 1;
        }
    }
    count
}

fn scramble(board: &mut Vec<bool>) {
    const ITERATIONS: usize = 256;
    for _ in 0..ITERATIONS {
        let (first, second) = choose_swap_indices();
        board.swap(first, second);
    }
}
```

Conflicting mutability requirements!

Borrowing arrays

Borrowing arrays

- Split into disjoint chunks
- Squares look nice, rows are easier to implement

Borrowing arrays - How?

- How do we borrow only part of a `Vec<T>`?
- While we're at it:
- Do we even care that our board is represented as a `Vec<bool>`?
- Could it be an array? `[bool; N*N]`

What defines a contiguous sequence of elements on the hardware-level?

Slices

- Two ideas:

1. Use two pointers: `begin` and `end`
2. Use a pointer and a length: `begin`, `length`

- Rust calls this a slice

Slices

- Slices are built into the Rust language:
- `[T]` is a slice of elements of type `T`
- Internally represented as `begin` + `length`
- Slices are unsized types
- By definition: The size of the slice is a runtime parameter!
- Consequence: Can't pass raw slices as function parameters, store them in user-defined types etc.
- Use slices through borrows: `&[T]` and `&mut [T]`

Aside: Why `begin` + `length` instead of `begin` + `end`?

- GPT5 says:

  [...] The length form keeps the metadata compact: one pointer-sized value plus one integer. Two pointers double the number of pointer-sized values. **On 64-bit systems this saves 8 bytes per slice.**

- True or not?

Slices vs. `Vec<T>`

- Slices are like reduced vectors:

- What do we lose if we don’t store a `capacity`?

Slices are non-owning

- No `capacity` == No way of changing the size
- Therefore slices are non-owning types!
- This has a lot of benefits:
- Trivial to copy (all slices implement `Copy`)
- Meant for passing around data (they are always borrowed)
- Easy to reason about (no ownership struggles)
- They compund (slices of slices of slices...)
- They still need lifetimes though

Slices - Usage example

```
fn main() {
    let numbers: [u32; 4] = [1, 2, 3, 4];

    // Slicing by passing a range `a..b` to the [] operator:
    // `..` is the unbounded (i.e. full) range
    let numbers_slice: &[u32] = &numbers[..];

    // Slices behave like arrays:
    // They have a len() method and support the [] operator:
    println!("{}", numbers_slice.len());
    println!("{}", numbers_slice[0]);

    // We can even loop over slices:
    for number in numbers_slice {
        println!("{}", number);
    }
}
```

Our game with slices

```
fn count_whites(board: &[bool]) -> usize {
    // ...
}

fn scramble(board: &mut [bool]) {
    // ...
}

fn main() {
    let mut board = gen_board();

    let split_point = 4;
    let counting_region = &board[..split_point];
    let scrambling_region = &mut board[split_point..];

    // Not real syntax, imagine the two functions run in parallel!
    in_parallel! {
        scramble(scrambling_region);
        count_whites(counting_region);
    }
}
```

Our game with slices

```
1  fn count_whites(board: &[bool]) -> usize {
2      // ...
3  }
4
5  fn scramble(board: &mut [bool]) {
6      // ...
7  }
8
9  fn main() {
10     let mut board = gen_board();
11
12     let split_point = 4;
13     let counting_region = &board[..split_point];
14     let scrambling_region = &mut board[split_point..];
15
16     // Not real syntax, imagine the two functions run in parallel!
17     in_parallel! {
18         scramble(scrambling_region);
19         count_whites(counting_region);
20     }
21 }
```

Doesn't compile :(

```
cannot borrow `board` as mutable because it is also borrowed as immutable
use `.split_at_mut(position)` to obtain two mutable non-overlapping sub-slices
```

Fixed

```
fn count_whites(board: &[bool]) -> usize {
    // ...
}

fn scramble(board: &mut [bool]) {
    // ...
}

fn main() {
    let mut board = gen_board();

    let split_point = 4;
    let (counting_region, scrambling_region) = board.split_at_mut(split_point);

    // Not real syntax, imagine the two functions run in parallel!
    in_parallel! {
        scramble(scrambling_region);
        count_whites(counting_region);
    }
}
```

- Not the complete code, check out `chunks_mut` to split into disjunct rows!

Come up with a visual classification of these types: `Vec<T>`, `&[T]`, `&mut [T]`
Which core concepts can you isolate, and is there a gap for a type that we might not yet have covered in this classification?
10 minutes in groups (3-4)

An immutable owned dynamic array

- We might care for an `ImmutableVec<T>`
- Different from `[T; N]` because arrays need a size known at compile-time!
- A bit weird because the term `Vec` implies growing…
- If slices are just a pointer + a length, this would also be a pointer + a length
- Same memory layout, different semantics
- Rust has this as `Box<[T]>`

`Box<T>`

- An owned instance of `T` on the heap
- `let val1 = 42;`
- On the stack
- `let val2 = Box::new(42);`
- Same value but on the heap (`val2` holds a pointer)

`Box<T>` is a language item

- Special compiler support for `Box<T>`!

```
assert_eq!(
    size_of::<Box<i32>>(),   // 1 * word-size
    size_of::<*const i32>()
);
assert_eq!(
    size_of::<Box<[i32]>>(), // 2 * word-size
    size_of::<&[i32]>()
);
assert_ne!(
    size_of::<&[i32]>(),
    size_of::<i32>()
);
```

Conversions

Cloning `Box<T>`

- Traits make it easy to clone a `Box<T>`:

```
impl<T: Clone> Clone for Box<T> {
    fn clone(&self) -> Self {
        // Box behaves like a pointer, so *self == T
        // This behavior comes from the `Deref` trait
        Box::new((*self).clone())
        //       -------^-------
        // clone the *inner* value and put that onto the heap
    }
}
```

Strings

- To represent text through (dynamic) arrays, we need an encoding
- Rust uses UTF-8 (the best text encoding there is!)
- Problem: UTF-8 characters can have varying sizes
- Time for a guessing game!

What does it print?

```
1  println!("{}", "hello".len());
```

5

What does it print?

```
1  println!("{}", "hallö".len());
```

6

What does it print?

```
1  println!("{}", "👋".len());
```

4

What does it print?

```
1  println!("{}", "👋🏽".len());
```

8

Welcome to UTF-8!

Learn more **here**

Rust Strings

- A Rust `String` is basically a `Vec<u8>` with special semantics
- `String::len` returns the number of bytes, not the number of characters or graphemes
- All strings must be valid UTF-8, which prevents slicing at arbitrary positions:

```
let wavy_hand = "👋";
println!("{}", &wavy_hand[..5]); // This panics!
```

- There is also the primitive type `char`, which represents one Unicode scalar value (basically a character)
- `String::chars` gives you an iterator over all the characters in a string

Characters

```
1 for c in "👋🏽".chars() {
2     print!("{c} ");
3 }
```

This prints:

```
👋 🏽
```

String slices

- Just as we had `[T]` for general slices, there is `str` for string slices
- Like `[u8]` but always valid UTF-8!
- This is a common pattern in Rust:

Usage	Owning	Non-owning
Arbitrary type	`Vec<T>`	`[T]`
UTF-8 text	`String`	`str`
File system path	`PathBuf`	`Path`

...



Speaker notes